

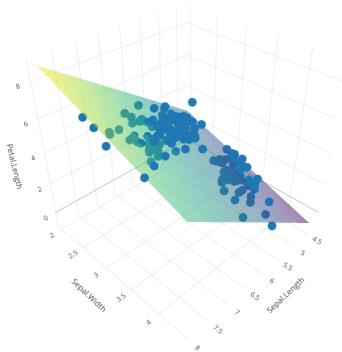
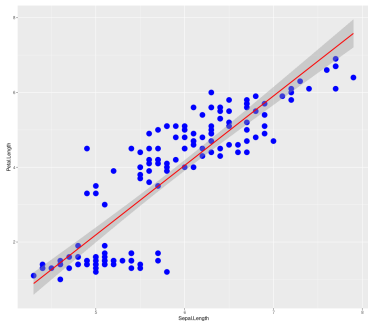
II: Modèles linéaires et généralisation

Guillaume Guex guillaume.guex@unil.ch

Les modèles linéaires

Les modèles linéaires

On parle de **modèles linéaires** lorsque la méthode utilisée suppose qu'il existe une **structure linéaire** dans nos données. P.ex., quand on suppose que les données suivent un **hyperplan** :



La **régression linéaire** est une méthode qui utilise un modèle linéaire sous-jacent. Elle suppose qu'une variable Y dépend linéairement des variables $X_1, \dots, X_p$:

$$Y = w_0 + w_1 X_1 + \dots + w_p X_p + E$$

où $w_0, w_1, \dots, w_p$ sont les **coefficients de régression** qui sont **inconnus**, et E un **bruit** (supposé suivre une loi normale).

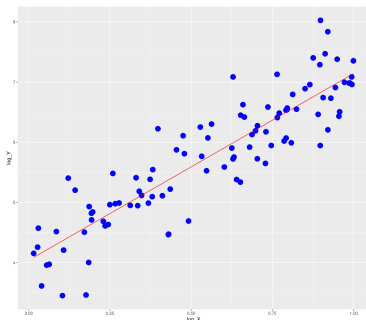
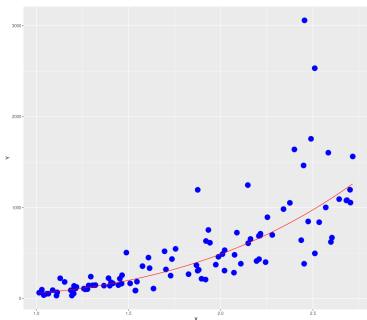
Pour un individu particulier i , on aura :

$$y_i = w_0 + w_1 x_{i1} + \dots + w_p x_{ip} + \epsilon_i$$

Parenthèse : la Features expansion

Attention, si l'on suppose un modèle linéaire entre des **fonctions de nos variables originales**, on reste dans les modèles linéaires. Par exemple le **modèle log-linéaire** suivant en fait partie :

$$\log(Y) = w_0 + w_1 \log(X) + E$$



On appelle cela l'**expansion des features (features expansion)**.

Régression linéaire : l'objectif

La régression linéaire est une méthode **supervisée de régression**. Sa tâche est d'obtenir les coefficients $w_0, w_1, \dots, w_p$, tels que les **prédictions** $\hat{y}_i$:

$$\hat{y}_i := w_0 + w_1 x_{i1} + \dots + w_p x_{ip}$$

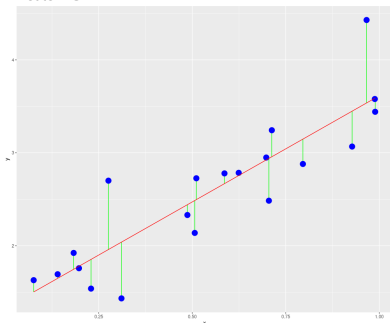
soient **proches de y_i pour tout individu i** .

Comment définir une **mesure** qui évalue la performance de la méthode sur la tâche ? On utilise l'**erreur carrée moyenne (Mean Squared Error)** :

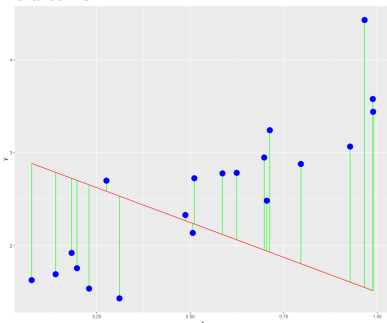
$$\begin{aligned} MSE(w_0, w_1, \dots, w_p) &= \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \\ &= \frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i)^2 \end{aligned}$$

Il s'agit donc de chercher les **paramètres** $w_0, w_1, \dots, w_p$ qui **minimisent** $MSE(w_0, w_1, \dots, w_p)$:

Petite MSE



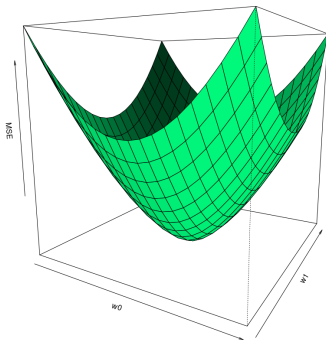
Grande MSE



$MSE(w_0, w_1, \dots, w_p)$ s'appelle une **fonction de perte ou d'erreur (loss or error function)**.

Perte vs Paramètres

Une représentation fréquente que nous allons faire durant ce cours est un **graphique de la fonction de perte vs les paramètres** :



Le but est de trouver le **minimum** de cette fonction. Dans le cadre de la régression linéaire, cette fonction est un parabololoïde.

Trouver les paramètres $w_0, w_1, \dots, w_p$ de la régression linéaire qui minimisent $MSE(w_0, w_1, \dots, w_p)$ est instantané, et se fait par un **calcul de dérivées**. On trouve

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

avec $\mathbf{w} = (w_k)$, $\mathbf{y} = (y_i)$ et $\mathbf{X}$ la matrice qui contient les (x_{ik}) avec une colonne de 1 ajoutée au début (pour w_0).

Cela ne sera pas toujours aussi simple...

La régression linéaire permet de résumer des **principes communs** à **plusieurs méthodes de ML** :

- On suppose que nos données suivent un **modèle** particulier, qui s'exprime via des **paramètres** (modèle linéaire).
- On pose une **fonction de perte** qui définit un objectif (MSE).
- On trouve les **paramètres qui minimisent la fonction de perte en fonction de nos données** (calcul direct).

Sous et surapprentissage

Famille de modèles : hyperparamètres

Supposons que nous avons 2 variables X et Y . On peut s'intéresser à la **famille de modèles linéaires** suivant :

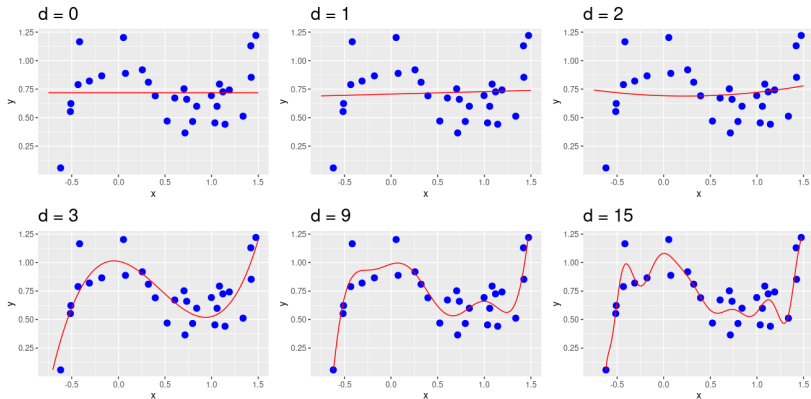
M(0)	$Y = w_0 + E$
M(1)	$Y = w_0 + w_1X + E$
M(2)	$Y = w_0 + w_1X + w_2X^2 + E$
$\vdots$	$\vdots$
M(d)	$Y = w_0 + w_1X + w_2X^2 + \dots + w_dX^d + E$

En réalité, on peut voir cette famille comme un seul modèle possédant un **hyperparamètre d** :

$$M(d) : Y = \sum_{q=0}^d w_q X^q + E$$

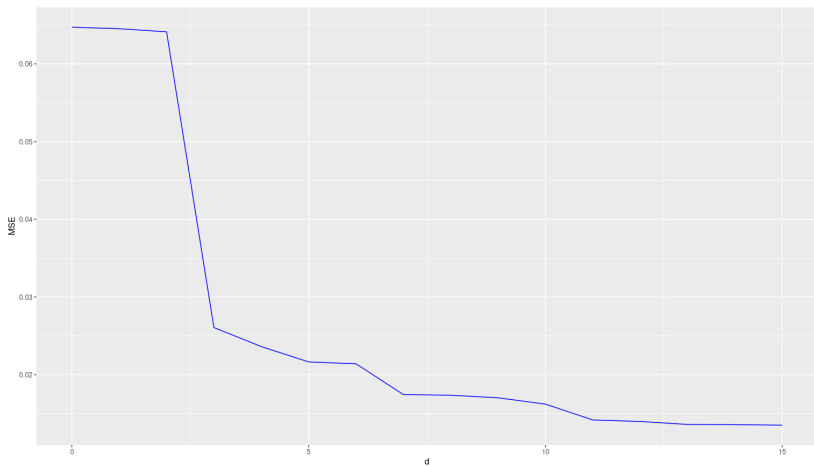
Famille de modèles : hyperparamètres

Avec un exemple de données, voici ce que le modèle va estimer (en minimisant la MSE), en fonction de l'**hyperparamètre d** :



Famille de modèles : hyperparamètres

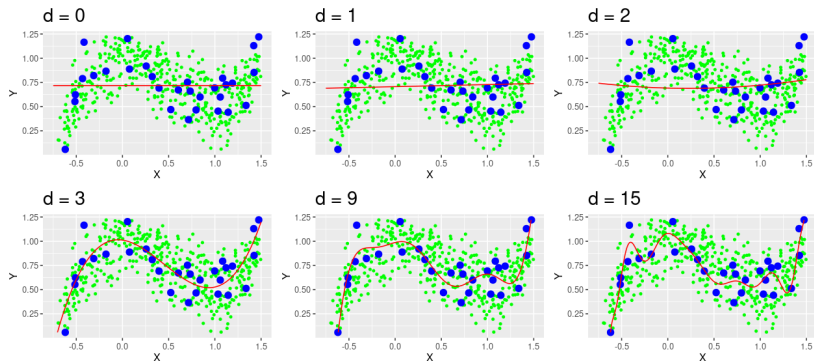
La fonction objectif MSE ne fait que **diminuer en fonction de d** :



Famille de modèles : hyperparamètres

En réalité, ces données viennent du modèle génératif

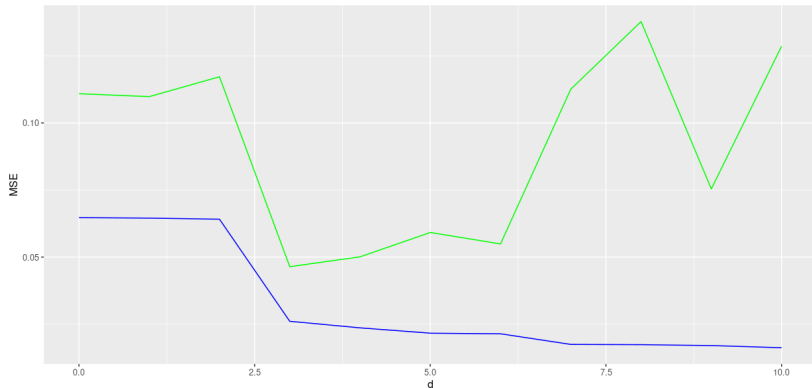
$$Y = X^3 - 1.5X^2 + 1 + E$$



Les modèles avec d trop élevés sont en **surapprentissage** (**overfitting**), ceux avec un d trop petits en **sousapprentissage** (**underfitting**).

Famille de modèles : hyperparamètres

En regardant la *MSE* par rapport au nouveaux points (en verts), on voit clairement l'**hyperparamètre idéal** à $d = 3$:



Le phénomène de sous et surapprentissage peut être observé en **séparant** notre jeu de données en deux groupes :

- Le jeu d'**entraînement** ($\sim 80\%$ des données) va être utilisé pour **ajuster les paramètres** du modèle.
- Le jeu de **test** ($\sim 20\%$ des données) va être utilisé pour **estimer l'erreur (la perte) de généralisation**.

On tâchera d'**échantillonner** tout en **conservant la structure** du jeu de données dans les sous-groupes (p.ex. même proportion d'individus dans chaque classe).

La régularisation

Nombre de variables excessif

Notons que phénomène de surapprentissage peut aussi arriver lorsque le **nombre de variables est trop élevé par rapport au nombre d'individus**. P.ex., le jeu de données `mtcars` (<https://gist.github.com/seankross/a412dfbd88b3db70b74b>) :

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
...											

possède 32 individus pour 11 variables. En effectuant une régression pour prédire `mpg` par rapport à toutes les autres variables, nous serons dans un régime de **surapprentissage**.

Comment choisir les variables à omettre ?

On pourrait effectuer un processus de sélection de variables (cf. MQIII), mais on peut également utiliser une procédure de **régularisation**.

Les procédures de régularisation sont nombreuses et permettent d'**éviter le surapprentissage**. On en distingue 2 catégories :

- La **régularisation explicite** consiste à **ajouter un terme à la fonction de perte**, afin que celle-ci évite de trouver des paramètres surajustés aux données.
- La **régularisation implicite** consiste en toutes les autres formes de régularisation, qui utilisent des moyens algorithmes pour arriver au même résultat (on en verra plus tard).

La régression Ridge et Lasso

On trouve fréquemment deux formes de régularisation explicite avec la régression linéaire :

$$\text{Ridge : } MSE_{L_2}(\lambda, w_0, \dots, w_p) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{k=0}^p w_p^2$$

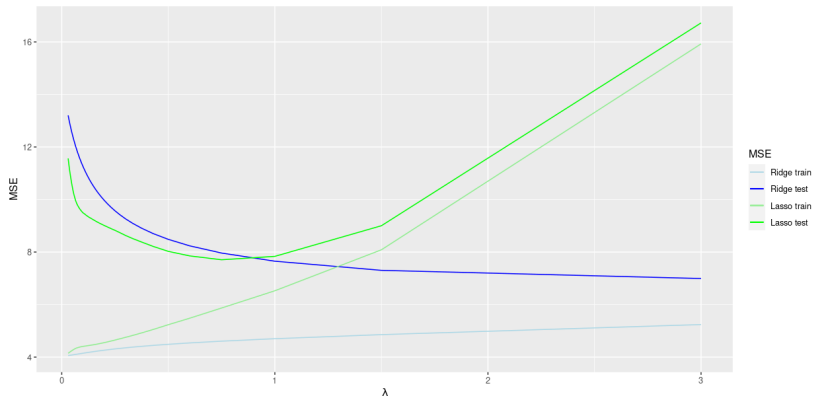
$$\text{Lasso : } MSE_{L_1}(\lambda, w_0, \dots, w_p) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{k=0}^p |w_p|$$

L'**hyperparamètre** λ permet de mettre de l'importance sur ce nouveau terme, et donc de la force à la régularisation.

Notez que la régression **Lasso a tendance à annuler certains coefficients de régression, Ridge à tous les diminuer.**

Les paramètres optimaux restent uniques et s'obtiennent facilement (mais avec une autre méthode pour la régression Lasso).

La régression Ridge et Lasso



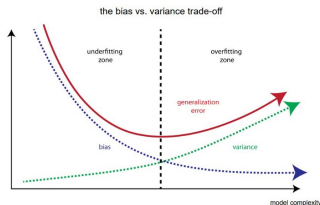
Lasso avec $\lambda = 0.75$:

$$\text{mpg} \simeq 36.17 - 0.09\text{cy1} - 0.01\text{hp} - 2.74\text{wt}$$

La complexité

On peut montrer que :

$$\text{Erreur de généralisation} = \text{biais}_{\text{modèle}}^2 + \text{variance}_{\text{modèle}} + \text{bruit}$$



Un modèle **peu complexe** est robuste aux variations des données (-variance), mais ajuste mal le jeu d'entraînement (+biais).

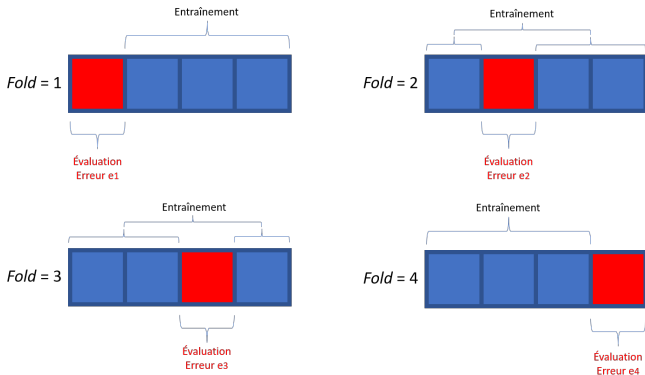
Un modèle **complexe** est sensible aux variations des données (+variance), mais ajuste bien le jeu d'entraînement (-biais).

Source : <https://towardsdatascience.com/bias-and-variance-but-what-are-they-really-ac539817e171>

Cross-validation et sélection de modèles

La cross-validation

La **k-fold cross-validation** est une procédure qui permet d'avoir une **estimation robuste de l'erreur de généralisation**. Cela consiste à séparer notre jeu de données en k parties égales, puis de faire k séries d'entraînement/test :



$$\text{Erreur moyenne} = \frac{1}{4} (e_1 + e_2 + e_3 + e_4)$$

On peut s'en servir pour **sélectionner un modèle** parmi une collection, ou **ajuster** un hyperparamètre.

Par exemple, on se donne une régression Ridge avec $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$. Pour chacune de ces valeurs, on effectue une cross-validation puis on sélectionne le modèle avec la plus petite erreur de généralisation.

S'il existe **plusieurs hyperparamètres**, on peut soit effectuer une **recherche sur grille (toutes les possibilités des paramètres)**, soit une **recherche aléatoire (tirage de couples possibles)**.

Attention, lorsqu'une sélection de modèle s'opère, l'erreur donnée par le modèle retenu est **trop optimiste**. **La procédure de sélection a été « ajustée » à tout le jeu de données !**

On doit donc séparer notre jeu de données en :

- Le **jeu d'entraînement** ($\sim 70\%$) permet d'entraîner les modèles.
- Le **jeu de validation** ($\sim 15\%$) permet de sélectionner le meilleur modèle.
- Le **jeu de test** ($\sim 15\%$) permet d'évaluer l'erreur de généralisation du meilleur modèle.

La cross-validation peut être alors **imbriquée** (nested).